

How we faked it

Async event handlers as container morphisms:
composing shapes forwards while responses flow back

September 1, 2026

Abstract

The fake is not a type-level trick with a network attached. **The fake is the async event handler.** A container’s inhabitant $(p : P) \rightarrow R p$ is a handler woken by a prompt; a morphism between containers sends shapes *forward* and brings responses *back*; and **await** is what allows both directions to be written as one straight-line function, in a server that is simultaneously a client. This note is about that, and about what the type system does and does not contribute to it.

1 A container is an event handler

§3 of Ghani’s note observes that event-driven programming “has a feature that turns out to be exactly a dependent type”: which responses are appropriate depends on which event arrived. A container (P, R) is that feature written down — P the prompts, $R p$ the positions, the responses appropriate to p — and an inhabitant of the interface is the dependent function $(p : P) \rightarrow R p$.

In this project that inhabitant is not a metaphor. It is the handler:

```
serveContainer({
  name: "agri", port: 8802,
  parse: (u: unknown) => AgPrompt | null,      // the prompt arrived as bytes
  answer: (p: AgPrompt, depth) => Promise<...>, // <-- THIS is (p : P) -> R p
});
```

answer is woken by a socket, receives one prompt, and owes exactly the response type that prompt indexes. Five such handlers are running while the game is played.

2 Two directions

A morphism of containers $(P, R) \rightarrow (Q, T)$ is a pair:

$$u : P \rightarrow Q \qquad f : (p : P) \rightarrow T (u p) \rightarrow R p$$

u turns my prompt into one for the level below — **shapes travel forwards**. f turns their response into mine — **responses travel back**. Control descends; data ascends. In a game about a command economy that is not a coincidence: Gosplan does not reap anything. It converts “feed the towns” into “Narkomzem: harvest”, and converts the reply back into a number for its own ledger.

The tower prints both directions as it runs. One **Sow**, across four processes:

```
:8801 |- gosplan   Sow order={"procurement":"firm","labour":{...
:8801 |   |- gosplan ((agri (x) smelter) (x) transport)
:8802 |       |- agri   Harvest workforce={"fields":97,...} tractors=0 year=1928
:8802 |       -| agri   {grain, grade, perWorker, withoutTractors}
:8803 |       |- industry Smelt workers=16 millCapacity=8 year=1928
:8803 |       -| industry {steel, idleCapacity}
:8804 |       |- transport Lay workers=7 steel=10 year=1928
:8804 |       -| transport {added, steelUsed}
:8801 -| gosplan   {kind, year, harvest, steel, ...}
```

$|-$ is a shape going forward, $-|$ a response coming back. Every indent is a morphism.

3 await is the hinge

Here is the whole of a morphism, in code:

```
const reply = await lower.run(u(prompt)); // shapes forward
return f(prompt, reply);                 // responses back
```

Both directions, one function body, read top to bottom. That is what `await` buys, and it is the fake. Remove it and there are two options, both of which break something:

Callbacks invert the control flow. Written `lower.run(u(prompt), (reply) => ...)`, the second half of the morphism is no longer the continuation of the first — it is a separate function handed downwards. *f* can no longer be *returned*; it must be *passed*. Two morphisms composed become a callback inside a callback, and the composite is no longer expressible as a morphism at all. The algebra survives in the types and dies in the code.

Blocking seizes the runtime. Gosplan is both a server and a client: woken by a socket, and while handling that prompt it delegates to four commissariats. If those downward calls block — `outputSync` rather than `await` — they block the whole isolate. The server cannot accept another connection while it waits for its own subordinates. A blocking handler that delegates is not merely slower; it has stalled the event loop it lives in.

So this is the substitution. `async/await` is what lets a morphism be an ordinary function in a program that is simultaneously listening and calling. Shapes compose forwards through *u*, control suspends at the `await`, and responses compose backwards through *f* — with the whole chain written as though it were sequential code, which is exactly what the algebra assumes it is.

4 The combinators, in both directions

Each combinator is a way of composing the forward flow, and each has a matching rule for the backward one.

	shapes forward	responses back
⊠	a prompt to each side, dispatched together	a reply from <i>each</i> ; the position is a pair
×	a prompt to each side	a reply from <i>exactly one</i>
+	a prompt to one side	that side's position, unchanged
◁	the second prompt is <i>built from the first reply</i>	a pair of replies

⊠ — forward together

```
const [left, right] = await Promise.all([a.run(s.left, d), b.run(s.right, d)]);
return { left, right };
```

Both shapes existed before the call, so neither can depend on the other's reply — independence enforced by the shape type `{left, right}`, not by the scheduler. The `Promise.all` is concurrency; a synchronous tensor firing its two hops in sequence is the same combinator, and slower.

◁ — where the two directions interleave

This is the one that needs the hinge, and the reason the sequential composition is the hard combinator:

```
const first = await a.run(s.first, depth); // forward, then BACK
const second = await b.run(s.next(first), depth); // the reply BECOMES the
                                                    // next shape: forward again
```

`s.next(first)` is the whole content of `◁`. A response flowing back turns around and becomes a shape flowing forward. The composite's shape is therefore not a pair of prompts but a prompt *together with a function* from its replies to the next prompt:

```
step<SA extends Shape<A>, SB extends Shape<B>>>(
  first: SA,
  next: (r: PosAt<A, SA>) => SB,           // SA inferred from 'first', so 'next'
)                                           // is typed at THAT fibre alone
```

In the game: what you may *sell* depends on what the railway actually *moved*, and the trade delegation cannot be asked anything until the haul has answered.

5 The tower makes the chain real

u is not a function call. It is a CLI tool:

```
delegate.ts <port> <json-prompt> <depth>      # stdout is the reply
```

It knows nothing about any container — port, JSON, depth, and it carries the text across. That ignorance is the point: *u* is a map of shapes, and shapes are what survive a wire. A CLI tool calls a CLI tool, whose server calls another, and the responses compose back up the same chain; the `depth` header is what indents the trace above.

Why processes rather than function calls. Each commissariat holds a hidden efficiency and a bias — how much it inflates a shortfall — and the player plans against the reports, not the truth. Inside one process that privacy is a convention, one stray import from collapsing. Across five processes with only JSON between them the operating system enforces it: there is no reference to reach through. The game’s subject is an information asymmetry, and the wire is what makes it a fact rather than a promise, at about a fifth of a second per hop.

6 What the type system contributes, and what it cannot

The handler is the fake; the types are what stop it lying about itself. Three mechanisms, and one hard limit.

The container as its graph

R cannot be a function from values to types, so a container is presented as the union of its fibres — over a *finite* index this is an equality:

$$\Sigma(s : S). B s = \bigcup_{s \in S} \{s\} \times B s$$

```
interface Fib<S, P> { readonly shape: S; readonly pos: P }
type PosAt<C extends Cont, S> =
  C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) : never;
```

`Fib` is never constructed; it is taken apart by `infer`. The handler’s signature then reads `run<S extends Shape<C>>(shape: S): Promise<PosAt<C, S>` — ask for a harvest, be handed a harvest report, uncast.

The hard limit, which is not TypeScript’s fault

The general definition, as it appears in Agda,

```
record Container : Shape : Set,   Pos : Shape → Set
```

requires a function from *values* to *types*. That is a genuine dependent function and it is not available in TypeScript — nor in Haskell, where `data Container = Container {shape :: Type; pos :: shape -> Type}` is rejected outright: a value-level field cannot be named in a type

position. The canonical container is the list functor, $\text{Shape} = \mathbb{N}$ and $\text{Pos } n = \text{Fin } n$; writing it needs one constructor per natural number in either language.

So containers do need dependent types, and neither language has them. What both can do is the *finite* case, by presenting a container as its graph — unions of fibres read by conditional types in TypeScript, a GADT indexed by its response type in Haskell. Every container in this project has finitely many prompts, which is the assumption the whole encoding rests on and the reason it works at all.

The bill

Ten casts, two per combinator, because every `run` body must destructure a shape it knows only through a conditional type and re-assert the result. All ten are inside `lib/algebra.ts`; none is in the game. Haskell’s version needs none, because pattern matching on a GADT genuinely refines the type rather than asserting it.

Two silent traps, both costly: the fibre test written `Extract<C, Fib<S, unknown>` instead of `S extends S1` makes every position **never**; and a type parameter occurring only inside conditional types is not an inference site, so `seqC(a, b)` silently infers `A = Cont` unless `Runner` carries a phantom `readonly fibres?: C`.

Summary

The fake	an <code>async</code> event handler: $(p : P) \rightarrow Rp$ woken by a socket
What it buys	shapes compose <i>forwards</i> through u and responses compose <i>backwards</i> through f , in one straight-line function — in a program that is listening and calling at the same time
Why not callbacks	control inverts; f must be passed rather than returned, and composites stop being morphisms
Why not blocking	a handler that delegates and blocks stalls the event loop it lives in
The hard combinator	$\triangleleft$, where a response turns around and becomes the next shape
What types add	the response type follows from the prompt, uncast, at ten casts confined to one library
What nothing adds	the general $\text{Pos} : \text{Shape} \rightarrow \text{Set}$. Finite containers only — in TypeScript and in Haskell alike
